Mars Storage System

Distributed Systems Project Report

Arduini Daniel 248727
Tezza Michael 248725

March 18, 2026

1 Structure of the Project

The project is built using Java and the Akka framework. At a macroscopic level, the system architecture is composed of three distinct types of entities:

- **Manager:** This entity acts as the system administrator. It sits entirely outside the peer-to-peer network and controls the lifecycle of the system. It is responsible for spawning new nodes, gracefully removing them, simulating sudden hardware crashes, and recovering nodes. It essentially offers an interface to orchestrate the network topology.
- **Clients:** These entities simulate external users or services interacting with the storage system. Multiple clients can exist simultaneously. They do not know the internal topology of the ring; they simply connect to any available node and issue `get` and `update` commands.
- **Nodes:** The core components of the system. Nodes are Akka actors organized in a logical circular ring. They route messages, manage the distributed storage, and coordinate with one another to ensure data replication and fault tolerance. Each node plays three roles simultaneously: a *coordinator* that handles client requests, a *replica* that stores data and responds to quorum messages, and a *network participant* that manages ring topology changes. These roles are implemented as separate logic classes (`CoordinatorLogic`, `ReplicaLogic`, `NetworkLogic`) that share a single `NodeContext` state object.

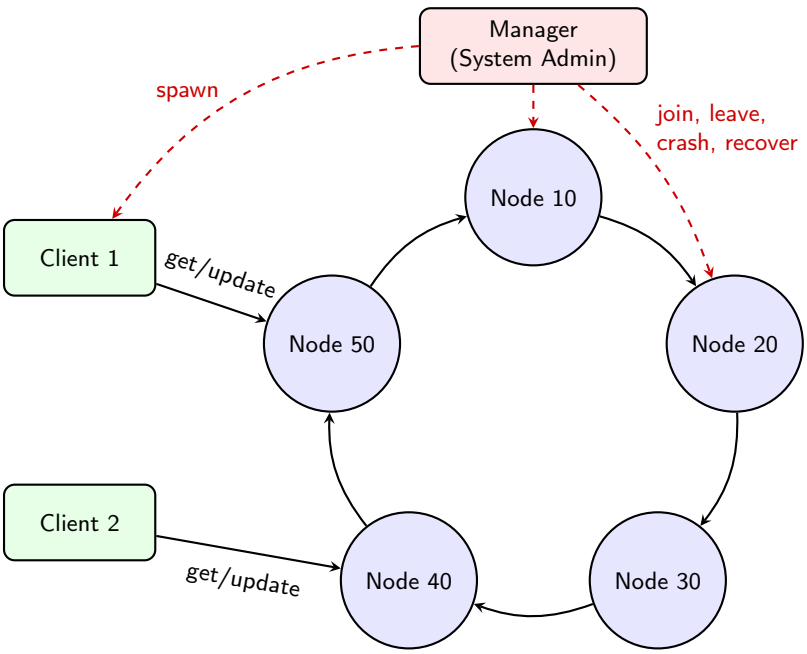

Figure 1: High-level architecture of the Mars Storage System.

2 Main Design Choices

2.1 Tracking Ongoing Operations

To handle the asynchronous nature of a distributed system, the architecture avoids blocking threads while waiting for network responses. Instead, ongoing operations are tracked using stateful context objects stored in the node's local memory.

This stateful tracking mechanism is utilized for both external Client Read/Write requests and internal Join/Recovery synchronizations.

When a node coordinates a transaction—either on behalf of an external client or on behalf of itself during a reboot—it generates a unique transaction identifier (`UUID`) and creates a temporary tracking object. This object holds essential information: a reference to the requester, the target data key, and a dynamic collection of the responses arriving from the replicas.

These tracking objects are stored in in-memory `HashMap`s. For external client operations, the coordinator simultaneously initiates an asynchronous timer based on a predefined timeout threshold T.

As acknowledgment messages arrive from the network, the node uses the transaction identifier to retrieve the corresponding tracking object, updates its internal counters, and evaluates whether the required Read or Write Quorum has been satisfied.

Once the quorum is reached, the transaction is finalized: if it was a client request, the appropriate response is sent back to the user; if it was an internal background read for a joining or recovering node, the node safely commits the verified data to its own disk. The tracking object is then deleted from memory. If the asynchronous timer expires before the quorum is achieved, the operation is automatically aborted, the tracking object is cleaned up, any locks held on replicas are explicitly released via `UnlockKeyMsg`, and the client is notified of the timeout, ensuring the system never hangs indefinitely.

2.2 Main Messages Used in the System

To keep the code simple and safe, all messages in the system are implemented as Java `Records`. We can group them into three main categories based on their purpose:

- **Client Messages:** Used for communication between the user and the system.
  - `ClientGetRequestMsg` / `ClientUpdateRequestMsg`: Sent by the client to ask the system to read or write data.
  - `ClientGetResponseMsg` / `ClientUpdateResponseMsg`: Sent back to the client by the coordinator to deliver the final result (success, failure, or data).
- **Internal Storage Messages:** Used between the coordinator and the replicas to manage quorums and locks.
  - `ReplicaReadRequestMsg` / `ReplicaWriteRequestMsg`: The coordinator sends these to ask replicas for data or to save new data. `ReplicaReadRequestMsg` carries an `intentToWrite` flag: when `true`, the replica acquires an exclusive lock on the key before replying.
  - `ReplicaReadResponseMsg` / `ReplicaWriteResponseMsg`: The replicas reply with the requested data or confirm the save. The read response also carries a `lockDenied` flag to signal that the key is currently held by another write operation.
  - `UnlockKeyMsg`: Used to free a locked key. Sent by the coordinator both on write completion and on timeout, and also fired by an internal per-replica timer to prevent indefinite locking if the coordinator fails mid-write.
- **Ring Management Messages:** Used to update the network shape and move data without losing it.
  - **Topology Updates:** `JoinMsg`, `NodeListRequestMsg` / `NodeListResponseMsg`, and `JoinedNetworkMsg` coordinate the addition of a new node and the distribution of the updated network view. `LeaveMsg` and `NodeLeavingMsg` handle graceful removal.
  - **Data Transfers:** `StorageRequestMsg` / `StorageResponseMsg` allow a joining or recovering node to fetch its initial dataset from its clockwise neighbour. `NodeLeavingMsg` carries the orphaned data that the leaving node hands off to newly responsible peers.

2.3 Simulating Crash and Recovery

In a real distributed system, if a server loses power, its active memory (RAM) is cleared, but the data on its physical hard drive is kept safe. To simulate this correctly in the Akka framework, we separate temporary memory from persistent storage.

1. Simulating a Crash. When the Manager wants to simulate a hardware failure, it sends a `CrashMsg` to a specific node. The node immediately executes `actorContext.stop(self)`, completely destroying the Akka actor. All temporary state—pending quorum counters, locks, and in-flight messages—is instantly erased, correctly simulating a sudden power loss.

2. Simulating a Physical Hard Drive. To ensure that saved data survives the crash, we use a `static ConcurrentHashMap` named `persistentDisks`. Because this map is `static`, it exists outside the lifecycle of any single actor. It acts like a warehouse of physical hard drives keyed by node ID. When an actor is destroyed, its entry in this map remains intact.

3. The Recovery Process. When the Manager reboots a crashed node, it spawns a new Akka actor with the same ID and sends it a `RecoverMsg`. Recovery proceeds in three steps:

- **Mounting the Disk:** The new actor retrieves its entry from `persistentDisks` and immediately has access to all data it held before the crash.
- **Updating the Topology:** The node contacts a bootstrap peer to obtain the current network membership list.
- **Synchronizing Missed Data:** The recovering node compares the keys from its clockwise neighbour's storage against its own disk. For every key it is now responsible for, it launches a background quorum read. Items no longer under its responsibility (because new nodes joined while it was down) are discarded. This guarantees the node holds exactly the right data—and the most recent version of it—before it resumes serving client requests.

2.4 Network Propagation Delay Simulation

To emulate realistic network conditions, every unicast message in the system is sent through a `sendWithDelay` helper defined in `BaseLogic`. Before each transmission, the node draws a random delay uniformly distributed between 10 ms and 50 ms and schedules the delivery via the Akka scheduler. This applies to all message types: client responses, quorum requests, topology updates, and data transfers. The simulation therefore exercises timeout and quorum logic under realistic asynchronous conditions rather than in a synchronous, zero-latency environment.

3 Discussion of the Implementation

3.1 Sequential Consistency

The system provides Sequential Consistency: all clients observe operations in the same global order, and no client ever reads a value older than one it has already seen.

The guarantee rests on three cooperating mechanisms.

Write serialisation via 2PL locking. Before committing a new value, a write coordinator first acquires an exclusive lock on the key at each of the N responsible replicas by sending a `ReplicaReadRequestMsg` with the `intentToWrite` flag set. A replica that already holds a lock for a different request immediately replies with `lockDenied`. If any replica denies the lock before the coordinator collects W grants, the write is aborted and the client is notified. This ensures that two writes to the same key can never interleave: one will always be fully serialised before the other starts committing.

Quorum intersection ($W + R > N$). With the parameters $N = 3$, $W = 2$, $R = 2$, every read quorum and every write quorum must share at least one replica ($W + R = 4 > N = 3$). Consequently, any read issued after a write has completed is guaranteed to contact at least one replica that already holds the new version. The coordinator returns the value with the highest version number among the R replies, so stale data from replicas that have not yet received the update is automatically discarded.

Read blocking during active writes. The same lock that prevents write/write conflicts also blocks concurrent reads. When a replica holds a lock with `intentToWrite = true`, it replies with `lockDenied` to any other read or write request for that key, regardless of the requester's UUID. A read coordinator that receives a `lockDenied` immediately aborts and notifies the client. This prevents a client from observing a

partially written value and then failing to observe it again on a subsequent read, which would violate sequential consistency.

Together, these three mechanisms ensure that the result of any execution is equivalent to some valid serial order of all operations, and that the operations issued by each individual client appear in their program order within that global sequence.

3.2 Message Efficiency and Data Routing

The system is designed to use the minimum number of network messages.

Read and write efficiency. The coordinator calculates the N responsible nodes locally (every node holds a complete view of the ring) and contacts only those nodes, avoiding any broadcast. A read completes in one round trip: the coordinator sends N requests and returns to the client as soon as R replies arrive. A write uses two round trips: one to lock and read the current version, one to commit the new value.

Join and leave efficiency. When a node joins, it contacts only its clockwise neighbour to obtain an initial dataset, then performs background quorum reads only for the keys it is responsible for—not for all keys in the system. When a node leaves, it computes the difference between the current and future responsible sets for each of its keys and sends data *only* to nodes that do not already hold a replica. Nodes that were already replicas before the departure receive only a topology notification with an empty data payload.

3.3 The Write and Lock Workflow

The two-phase write protocol is the core algorithm guaranteeing consistency.

- **Phase 1 — Acquire locks and read version.** The coordinator sends `ReplicaReadRequestMsg` (with `intentToWrite = true`) to all N responsible replicas. Each replica that is not already locked sets a lock entry keyed by the request's UUID, starts an internal safety timer (set to $T + 500\,\text{ms}$), and returns its stored value and version. If a replica is already locked, it immediately replies `lockDenied`.
- **Conflict handling.** If any `lockDenied` arrives before the coordinator has collected W successful lock grants, the write is aborted: the coordinator removes the pending context, notifies the client of failure, and broadcasts `UnlockKeyMsg` to all N replicas to release any partial locks already granted.
- **Phase 2 — Commit and unlock.** Once W lock grants have been collected, the coordinator determines the maximum version among the replies, increments it by one, and notifies the client of success immediately. It then sends `ReplicaWriteRequestMsg` with the new `StorageData` to all N replicas as a fire-and-forget operation. Each replica writes the value (keeping whichever version is higher, via merge) and releases its lock. The coordinator removes the pending context and starts the next queued write for the same key only after receiving W write acks, ensuring that the locks are fully released before a subsequent write begins.
- **Timeout and deadlock prevention.** If the coordinator does not receive W phase-1 grants before the timeout T expires, it removes the pending context, informs the client of failure, and sends `UnlockKeyMsg` to all N replicas to release any partial locks already granted. The per-replica safety timer ($T + 500$ ms) serves as a final safeguard in case the coordinator itself crashes mid-write.

3.4 Additional Assumptions

The following assumptions were made during the design and implementation of the system.

- **Persistent storage survives crashes.** A software or hardware crash does not destroy the data on the node's logical disk. The static `persistentDisks` map models this assumption.
- **Unique node IDs.** The Manager assigns a strictly unique integer ID to every node. No two nodes share the same position on the ring.